

# Neural Networks: From Motivation to Practice

BINF 4002 — Machine Learning for Health

---

Matthew McDermott

Department of Biomedical Informatics, Columbia University

1. **Motivation** — why neural nets?
2. **Anatomy** — layers, activations, depth
3. **Layer Types** — architecture as bias
4. **Loss Functions** — matching task to objective
5. **Optimization** — the practical toolkit
6. **Putting It Together** — recipe & visualizations

All experiments run live on MNIST in the companion notebook.

❖ **Notebook:** `nn_fundamentals_lab.ipynb`

# Motivation

---

# What Do We Need From a Function Class?

We want a **parametric function**  $f_{\theta}(\mathbf{x})$  that is simultaneously:

# What Do We Need From a Function Class?

We want a **parametric function**  $f_{\theta}(\mathbf{x})$  that is simultaneously:

1. **Arbitrarily expressive** — can approximate any continuous mapping.

# What Do We Need From a Function Class?

We want a **parametric function**  $f_{\theta}(\mathbf{x})$  that is simultaneously:

1. **Arbitrarily expressive** — can approximate any continuous mapping.
2. **Parametric** — fixed-size  $\theta$ ; doesn't grow with dataset.

# What Do We Need From a Function Class?

We want a **parametric function**  $f_{\theta}(\mathbf{x})$  that is simultaneously:

1. **Arbitrarily expressive** — can approximate any continuous mapping.
2. **Parametric** — fixed-size  $\theta$ ; doesn't grow with dataset.
3. **Efficiently auto-differentiable** —  $\nabla_{\theta}\mathcal{L}$  computable in  $O(\text{forward pass})$ .

# What Do We Need From a Function Class?

We want a **parametric function**  $f_{\theta}(\mathbf{x})$  that is simultaneously:

1. **Arbitrarily expressive** — can approximate any continuous mapping.
2. **Parametric** — fixed-size  $\theta$ ; doesn't grow with dataset.
3. **Efficiently auto-differentiable** —  $\nabla_{\theta}\mathcal{L}$  computable in  $O(\text{forward pass})$ .

## Why all three?

Health data is high-dimensional and nonlinear (expressiveness), models must be deployable (parametric), and we need scalable optimization (autodiff).

# The Landscape of Alternatives

| Approach      | Universal? | Parametric? | Autodiff? |
|---------------|------------|-------------|-----------|
| Linear models | ✗          | ✓           | ✓         |

# The Landscape of Alternatives

| Approach      | Universal? | Parametric?       | Autodiff? |
|---------------|------------|-------------------|-----------|
| Linear models | ✗          | ✓                 | ✓         |
| Kernel / GP   | ✓          | ✗ (grows w/ data) | ✗         |

# The Landscape of Alternatives

| Approach        | Universal? | Parametric?        | Autodiff?           |
|-----------------|------------|--------------------|---------------------|
| Linear models   | ✗          | ✓                  | ✓                   |
| Kernel / GP     | ✓          | ✗ (grows w/ data)  | ✗                   |
| Trees / forests | ✓          | ~ (grows w/ depth) | ✗ (discrete splits) |

# The Landscape of Alternatives

| Approach        | Universal? | Parametric?        | Autodiff?           |
|-----------------|------------|--------------------|---------------------|
| Linear models   | ✗          | ✓                  | ✓                   |
| Kernel / GP     | ✓          | ✗ (grows w/ data)  | ✗                   |
| Trees / forests | ✓          | ~ (grows w/ depth) | ✗ (discrete splits) |
| Neural nets     | ✓          | ✓                  | ✓                   |

## Key insight

A neural network is a **composition of simple differentiable operations**.

The chain rule (autodiff) handles the rest.

# Anatomy of a Neural Network

---

# The Fundamental Building Block: A Layer

$$h = \sigma(Wx + b)$$

- $x \in \mathbb{R}^{d_{\text{in}}}$  — input
- $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$  — weights
- $b \in \mathbb{R}^{d_{\text{out}}}$  — bias
- $\sigma(\cdot)$  — nonlinear activation
- $h \in \mathbb{R}^{d_{\text{out}}}$  — output

Parameter count per layer:

$$d_{\text{out}} \times d_{\text{in}} + d_{\text{out}}$$

For  $784 \rightarrow 256$ : 200,960 parameters.

❖ **Lab §1** — ManualMLP class builds this with raw `nn.Parameter` tensors so you can see every matrix multiply.

## Worked Example: Computing a Layer Output

**A tiny layer:**  $d_{\text{in}} = 3$ ,  $d_{\text{out}} = 2$ , activation = ReLU.

$$\mathbf{W} = \begin{pmatrix} 0.5 & -1.0 & 0.3 \\ 0.2 & 0.8 & -0.4 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 0.1 \\ -0.2 \end{pmatrix}, \quad \mathbf{x} = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}$$

## Worked Example: Computing a Layer Output

**A tiny layer:**  $d_{\text{in}} = 3$ ,  $d_{\text{out}} = 2$ , activation = ReLU.

$$W = \begin{pmatrix} 0.5 & -1.0 & 0.3 \\ 0.2 & 0.8 & -0.4 \end{pmatrix}, \quad b = \begin{pmatrix} 0.1 \\ -0.2 \end{pmatrix}, \quad x = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}$$

**Step 1: Affine transform  $Wx + b$**

$$\begin{pmatrix} 0.5(1) + (-1.0)(2) + 0.3(3) \\ 0.2(1) + 0.8(2) + (-0.4)(3) \end{pmatrix} + \begin{pmatrix} 0.1 \\ -0.2 \end{pmatrix} = \begin{pmatrix} -0.6 \\ 0.4 \end{pmatrix} + \begin{pmatrix} 0.1 \\ -0.2 \end{pmatrix} = \begin{pmatrix} -0.5 \\ 0.2 \end{pmatrix}$$

## Worked Example: Computing a Layer Output

**A tiny layer:**  $d_{\text{in}} = 3$ ,  $d_{\text{out}} = 2$ , activation = ReLU.

$$W = \begin{pmatrix} 0.5 & -1.0 & 0.3 \\ 0.2 & 0.8 & -0.4 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 0.1 \\ -0.2 \end{pmatrix}, \quad \mathbf{x} = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}$$

**Step 1: Affine transform**  $W\mathbf{x} + \mathbf{b}$

$$\begin{pmatrix} 0.5(1)+(-1.0)(2)+0.3(3) \\ 0.2(1)+0.8(2)+(-0.4)(3) \end{pmatrix} + \begin{pmatrix} 0.1 \\ -0.2 \end{pmatrix} = \begin{pmatrix} -0.6 \\ 0.4 \end{pmatrix} + \begin{pmatrix} 0.1 \\ -0.2 \end{pmatrix} = \begin{pmatrix} -0.5 \\ 0.2 \end{pmatrix}$$

**Step 2: ReLU**  $\sigma(z) = \max(0, z)$        $\mathbf{h} = (\max(0, -0.5), \max(0, 0.2))^T = (0, 0.2)^T$

Neuron 1 is **dead** (ReLU kills the negative pre-activation). Neuron 2 passes through.

## Why the Nonlinearity is Essential

Without  $\sigma$ , composing layers **collapses to a single linear map**:

$$h_2 = W_2(W_1x + b_1) + b_2 = \underbrace{(W_2W_1)}_{\tilde{W}}x + \underbrace{(W_2b_1 + b_2)}_{\tilde{b}}$$

# Why the Nonlinearity is Essential

Without  $\sigma$ , composing layers **collapses to a single linear map**:

$$h_2 = W_2(W_1x + b_1) + b_2 = \underbrace{(W_2W_1)}_{\tilde{W}}x + \underbrace{(W_2b_1 + b_2)}_{\tilde{b}}$$

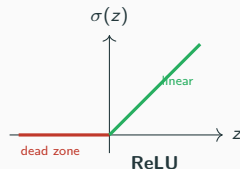
| Activation | Formula                         | Notes                                    |
|------------|---------------------------------|------------------------------------------|
| ReLU       | $\max(0, x)$                    | Default. Simple, sparse, fast.           |
| GELU       | $x \Phi(x)$                     | Smooth ReLU. Standard in transformers.   |
| Sigmoid    | $1/(1 + e^{-x})$                | Output $\in (0, 1)$ . Probability heads. |
| Tanh       | $(e^x - e^{-x})/(e^x + e^{-x})$ | Output $\in (-1, 1)$ . Centered sigmoid. |
| Softmax    | $e^{x_i} / \sum_j e^{x_j}$      | Produces a distribution over classes.    |

# Activations Impose Geometric Constraints

The choice of activation determines what the layer **can and cannot represent**.

## ReLU zeroes the negative half-space.

If all inputs are positive and all weights are positive, then  $\mathbf{W}\mathbf{x} + \mathbf{b} > 0$  and ReLU does nothing — the layer is effectively linear.



# Activations Impose Geometric Constraints

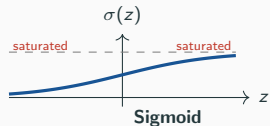
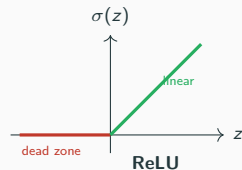
The choice of activation determines what the layer **can and cannot represent**.

## ReLU zeroes the negative half-space.

If all inputs are positive and all weights are positive, then  $\mathbf{W}\mathbf{x} + \mathbf{b} > 0$  and ReLU does nothing — the layer is effectively linear.

## Saturation kills gradients.

Sigmoid/tanh saturate for large  $|z|$ :  $\sigma'(z) \approx 0$ . If many neurons saturate, gradients vanish.



# Activations Impose Geometric Constraints

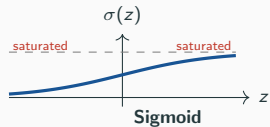
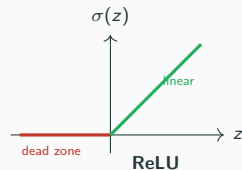
The choice of activation determines what the layer **can and cannot represent**.

## ReLU zeroes the negative half-space.

If all inputs are positive and all weights are positive, then  $\mathbf{W}\mathbf{x} + \mathbf{b} > 0$  and ReLU does nothing — the layer is effectively linear.

## Saturation kills gradients.

Sigmoid/tanh saturate for large  $|z|$ :  $\sigma'(z) \approx 0$ . If many neurons saturate, gradients vanish.



## Rule of thumb

**Default:** ReLU (or GELU for transformers). **Final layer:** sigmoid for probabilities, softmax for multi-class, identity for regression.

# Composing Layers: A Deep Network

$$\mathbf{h}_0 = \mathbf{x}$$

$$\mathbf{h}_\ell = \sigma_\ell(\mathbf{W}_\ell \mathbf{h}_{\ell-1} + \mathbf{b}_\ell) \quad \ell = 1, \dots, L$$

$$f_\theta(\mathbf{x}) = \mathbf{h}_L$$

## Universal Approximation (Cybenko '89, Hornik '91)

A single hidden layer of sufficient width can approximate any continuous function on a compact set.

# Composing Layers: A Deep Network

$$\mathbf{h}_0 = \mathbf{x}$$

$$\mathbf{h}_\ell = \sigma_\ell(\mathbf{W}_\ell \mathbf{h}_{\ell-1} + \mathbf{b}_\ell) \quad \ell = 1, \dots, L$$

$$f_\theta(\mathbf{x}) = \mathbf{h}_L$$

## Universal Approximation (Cybenko '89, Hornik '91)

A single hidden layer of sufficient width can approximate any continuous function on a compact set.

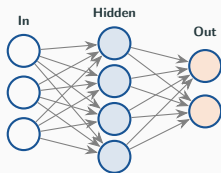
**But:** “sufficient width” may be **exponentially large**.

**Depth** is far more parameter-efficient than width in practice.

*Intuition:* each layer learns a new coordinate system in which the remaining problem is simpler.

# Reading Neural Network Diagrams

## Style 1: Neuron-level



Circles = neurons, lines = weights.  
Arrows show *whether* info flows (nonzero  $w$ ), not *how much*. A missing arrow =  $w = 0$  = no information flow through that path.

## Style 2: Block-level



Blocks = operations.  
This is what papers typically use.

## Style 3: Code-level

```
nn.Sequential(  
    nn.Linear(784, 256),  
    nn.ReLU(),  
    nn.Linear(256, 10),  
)
```

The diagram *is* the code.  
PyTorch reads like an architecture spec.

## Layer Types as Inductive Biases

---

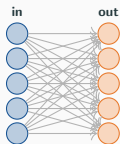
# Architecture Encodes Assumptions

| Layer Type         | Core Assumption                  | Operation                                                                                       |
|--------------------|----------------------------------|-------------------------------------------------------------------------------------------------|
| Fully connected    | No structural assumption         | $\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$                                        |
| Convolutional      | Local, translation-equivariant   | $h_i = \sigma(\sum_k w_k \cdot x_{i+k} + b)$                                                    |
| Recurrent<br>(RNN) | Sequential; hidden state summary | $\mathbf{h}_t = \sigma(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b})$ |
| Attention          | Set-like; learned interactions   | $\text{softmax}(\frac{QK^\top}{\sqrt{d_k}}) V$                                                  |

The choice of layer type determines what the model **can easily learn** vs. what it must **fight to learn**.

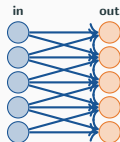
# How Layer Types Differ: Connectivity Patterns

## Fully Connected



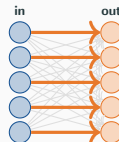
Every input connects to every output.  
No structural prior.

## Convolutional



Each output sees only a **local window**.  
Same weights reused at every position.

## Attention



Structurally fully connected, but weights are **learned & input-dependent**. Highlighted: each token's strongest self-attention.

# Mapping Architecture to Clinical Domain

| Data Modality          | Typical Architecture | Why                                      |
|------------------------|----------------------|------------------------------------------|
| Tabular / EHR features | MLP or Transformer   | No spatial/sequential structure          |
| Medical images         | CNN (ResNet, etc.)   | Spatial locality, translation invariance |
| Clinical notes         | Transformer          | Long-range dependencies                  |
| Vitals / waveforms     | CNN or Transformer   | Local temporal patterns                  |
| EHR event sequences    | Transformer          | Variable-length, irregular timing        |
| Molecular graphs       | GNN                  | Relational structure                     |

# Mapping Architecture to Clinical Domain

| Data Modality          | Typical Architecture | Why                                      |
|------------------------|----------------------|------------------------------------------|
| Tabular / EHR features | MLP or Transformer   | No spatial/sequential structure          |
| Medical images         | CNN (ResNet, etc.)   | Spatial locality, translation invariance |
| Clinical notes         | Transformer          | Long-range dependencies                  |
| Vitals / waveforms     | CNN or Transformer   | Local temporal patterns                  |
| EHR event sequences    | Transformer          | Variable-length, irregular timing        |
| Molecular graphs       | GNN                  | Relational structure                     |

## Rule of thumb

**Not sure?** Start with a Transformer — fewest structural assumptions.

**Small data?** MLP or logistic regression may outperform deep models.

# Loss Functions

---

## Choosing a Loss Function

The loss  $\mathcal{L}(f_{\theta}(\mathbf{x}), y)$  measures how wrong the model is. The choice depends on the **task**:

| Task            | Loss                                                   | Final act. |
|-----------------|--------------------------------------------------------|------------|
| Binary classif. | Binary CE: $-[y \log \hat{y} + (1-y) \log(1-\hat{y})]$ | Sigmoid    |
| Multi-class     | Cross-entropy: $-\sum_c y_c \log \hat{y}_c$            | Softmax    |
| Regression      | MSE: $(y - \hat{y})^2$                                 | Identity   |
| Survival        | Negative partial log-likelihood                        | Varies     |

# Choosing a Loss Function

The loss  $\mathcal{L}(f_{\theta}(\mathbf{x}), y)$  measures how wrong the model is. The choice depends on the **task**:

| Task            | Loss                                                   | Final act. |
|-----------------|--------------------------------------------------------|------------|
| Binary classif. | Binary CE: $-[y \log \hat{y} + (1-y) \log(1-\hat{y})]$ | Sigmoid    |
| Multi-class     | Cross-entropy: $-\sum_c y_c \log \hat{y}_c$            | Softmax    |
| Regression      | MSE: $(y - \hat{y})^2$                                 | Identity   |
| Survival        | Negative partial log-likelihood                        | Varies     |

## Unifying view: every loss above is a negative log-likelihood

Binary CE = NLL of a Bernoulli.

Cross-entropy = NLL of a Categorical.

MSE = NLL of a Gaussian (up to a constant).

Survival loss is already a partial NLL.

Choosing a loss  $\Leftrightarrow$  choosing a probabilistic model for  $p(y \mid \mathbf{x})$ .

# Optimization in Practice

---

# This Is Where Projects Succeed or Fail

Theory gives us SGD. Practice requires a **constellation of techniques**:

1. Initialization
2. Normalization
3. Residual connections
4. Optimizer choice
5. Learning rate & schedules
6. LR  $\leftrightarrow$  batch size relationship
7. Gradient clipping
8. Gradient accumulation
9. Monitoring (wandb)
10. Regularization

Each of the following slides covers one topic.

The  **lab callouts** point you to live experiments.

# 1. Why Does Initialization Matter?

**Goal:** at init, every layer should produce outputs with **roughly the same variance** as its inputs.

# 1. Why Does Initialization Matter?

**Goal:** at init, every layer should produce outputs with **roughly the same variance** as its inputs.

**But why?** Consider a deep network with  $L$  layers. If each layer scales its input variance by a factor  $\alpha$ :

$$\text{Var}(\mathbf{h}_L) = \alpha^L \text{Var}(\mathbf{x})$$

# 1. Why Does Initialization Matter?

**Goal:** at init, every layer should produce outputs with **roughly the same variance** as its inputs.

**But why?** Consider a deep network with  $L$  layers. If each layer scales its input variance by a factor  $\alpha$ :

$$\text{Var}(\mathbf{h}_L) = \alpha^L \text{Var}(\mathbf{x})$$

## The exponential amplification problem

$\alpha > 1$ : variance explodes exponentially with depth  $\rightarrow$  NaN activations/gradients.

$\alpha < 1$ : variance collapses exponentially  $\rightarrow$  all activations  $\approx 0$ , no learning.

$\alpha = 1$ : variance preserved across layers — **this is the target.**

# 1. Why Does Initialization Matter?

**Goal:** at init, every layer should produce outputs with **roughly the same variance** as its inputs.

**But why?** Consider a deep network with  $L$  layers. If each layer scales its input variance by a factor  $\alpha$ :

$$\text{Var}(\mathbf{h}_L) = \alpha^L \text{Var}(\mathbf{x})$$

## The exponential amplification problem

$\alpha > 1$ : variance explodes exponentially with depth  $\rightarrow$  NaN activations/gradients.

$\alpha < 1$ : variance collapses exponentially  $\rightarrow$  all activations  $\approx 0$ , no learning.

$\alpha = 1$ : variance preserved across layers — **this is the target.**

The right initialization sets  $\alpha \approx 1$  by choosing weight variance as a function of layer width.

# 1. Initialization — Strategies

**Xavier / Glorot** (tanh, sigmoid):

$$W_{ij} \sim \mathcal{N}\left(0, \frac{2}{d_{\text{in}} + d_{\text{out}}}\right)$$

**Kaiming / He** (ReLU):

$$W_{ij} \sim \mathcal{N}\left(0, \frac{2}{d_{\text{in}}}\right)$$

# 1. Initialization — Strategies

**Xavier / Glorot** (tanh, sigmoid):

$$W_{ij} \sim \mathcal{N}\left(0, \frac{2}{d_{\text{in}} + d_{\text{out}}}\right)$$

**Kaiming / He** (ReLU):

$$W_{ij} \sim \mathcal{N}\left(0, \frac{2}{d_{\text{in}}}\right)$$

## What goes wrong

**Too large** → activations/gradients explode

**Too small** → activations collapse to 0

**All zeros** → symmetry never broken

# 1. Initialization — Strategies

**Xavier / Glorot** (tanh, sigmoid):

$$W_{ij} \sim \mathcal{N}\left(0, \frac{2}{d_{\text{in}} + d_{\text{out}}}\right)$$

**Kaiming / He** (ReLU):

$$W_{ij} \sim \mathcal{N}\left(0, \frac{2}{d_{\text{in}}}\right)$$

## What goes wrong

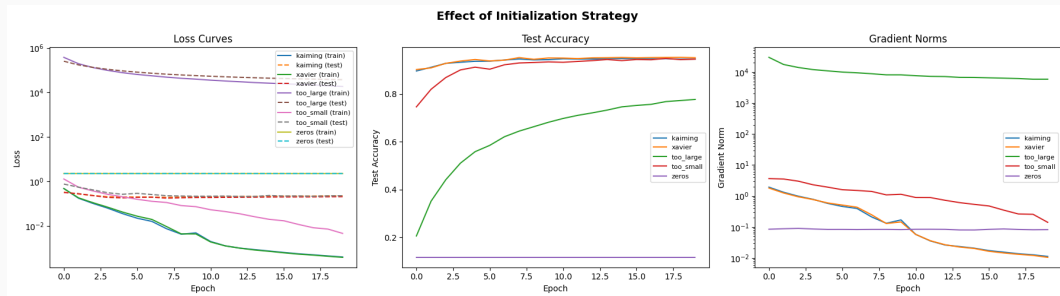
**Too large** → activations/gradients explode

**Too small** → activations collapse to 0

**All zeros** → symmetry never broken

**Why these formulas?** If  $\text{Var}(x_j) = v$  and weights are i.i.d. with variance  $\sigma^2$ , then  $\text{Var}(\sum_j W_{ij}x_j) = d_{\text{in}} \sigma^2 v$ . Setting  $\sigma^2 = 1/d_{\text{in}}$  (Xavier) or  $2/d_{\text{in}}$  (Kaiming, accounting for ReLU halving) keeps output variance  $\approx$  input variance.

# 1. Initialization — Lab Results

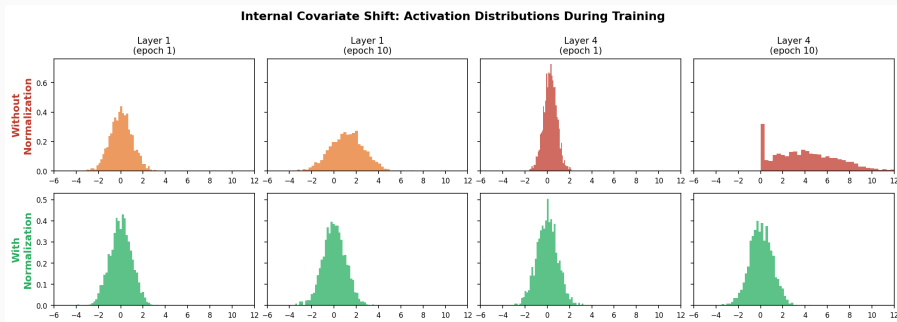


Kaiming and Xavier converge to  $> 95\%$  accuracy. Too-large init ( $\sigma=5$ ) causes exploding gradients ( $> 10^3$ , right panel). Too-small ( $\sigma=10^{-4}$ ) and zeros never escape their starting point — loss flatlines.

❖ **Lab §4** — Five strategies on the same 3-layer MLP / MNIST. Try it: swap kaiming  $\rightarrow$  zeros and watch accuracy flatline.

## 2. Normalization — The Problem

Even with good init, activation distributions **drift** during training — this is called **internal covariate shift**.



Without normalization, later layers must constantly re-adapt to shifting input distributions. Normalization forces each layer's inputs to have stable statistics.

## 2. Normalization — Methods

### Batch Normalization

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta$$

Normalize across the *batch* per feature.

$\gamma, \beta$  are learned.

Caveat: train/test behavior differs (running stats).

### Layer Normalization

$$\hat{x}_i = \frac{x_i - \mu_{\text{layer}}}{\sqrt{\sigma_{\text{layer}}^2 + \epsilon}}$$

Normalize across *features* per example.

No batch dependence.

Standard in transformers.

## 2. Normalization — Methods

### Batch Normalization

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta$$

Normalize across the *batch* per feature.

$\gamma, \beta$  are learned.

Caveat: train/test behavior differs (running stats).

### Layer Normalization

$$\hat{x}_i = \frac{x_i - \mu_{\text{layer}}}{\sqrt{\sigma_{\text{layer}}^2 + \epsilon}}$$

Normalize across *features* per example.

No batch dependence.

Standard in transformers.

### Rule of thumb

CNNs? BatchNorm (after conv, before activation).

Transformers? LayerNorm.

Small/variable batches? LayerNorm or GroupNorm.

## 2. Normalization — Methods

### Batch Normalization

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta$$

Normalize across the *batch* per feature.

$\gamma, \beta$  are learned.

Caveat: train/test behavior differs (running stats).

### Layer Normalization

$$\hat{x}_i = \frac{x_i - \mu_{\text{layer}}}{\sqrt{\sigma_{\text{layer}}^2 + \epsilon}}$$

Normalize across *features* per example.

No batch dependence.

Standard in transformers.

### Rule of thumb

CNNs? BatchNorm (after conv, before activation).

Transformers? LayerNorm.

Small/variable batches? LayerNorm or GroupNorm.

**Historical note:** Self-Normalizing Networks (Klambauer et al., 2017) used the SELU activation to maintain stable distributions without explicit normalization layers. Elegant in theory, but rarely used today — explicit normalization is simpler and more robust.

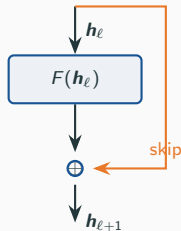
### 3. Residual Connections

**The degradation problem:** naively, deeper  $\neq$  better. A 20-layer plain net can train **worse** than a 5-layer net.

**The fix (He et al., 2015):**

$$\mathbf{h}_{\ell+1} = \mathbf{h}_{\ell} + F(\mathbf{h}_{\ell})$$

- Identity is the **default** (set  $F = 0$ ).
- Gradients flow directly through skip:  
 $\frac{\partial \mathbf{h}_L}{\partial \mathbf{h}_{\ell}} = I + \dots$
- Enabled 100+ layer training (ResNet).



❖ **Lab §5** — Depth experiment compares 3-layer baseline, 10-layer plain, + BN, + LN, + residual, + BN + residual.

### 3. Residual Connections — Deeper Interpretations

Beyond “gradient highway,” residual connections have richer interpretations:

### 3. Residual Connections — Deeper Interpretations

Beyond “gradient highway,” residual connections have richer interpretations:

- **Ensemble of paths** (Veit et al., 2016): A ResNet of  $L$  blocks has  $2^L$  paths from input to output. Empirically, most gradient signal flows through *short* paths — the network behaves like an ensemble of shallow networks.

### 3. Residual Connections — Deeper Interpretations

Beyond “gradient highway,” residual connections have richer interpretations:

- **Ensemble of paths** (Veit et al., 2016): A ResNet of  $L$  blocks has  $2^L$  paths from input to output. Empirically, most gradient signal flows through *short* paths — the network behaves like an ensemble of shallow networks.
- **Iterative refinement**: Each block computes a small *correction*  $F(\mathbf{h}_\ell)$  to the current representation, rather than building a new one from scratch.

### 3. Residual Connections — Deeper Interpretations

Beyond “gradient highway,” residual connections have richer interpretations:

- **Ensemble of paths** (Veit et al., 2016): A ResNet of  $L$  blocks has  $2^L$  paths from input to output. Empirically, most gradient signal flows through *short* paths — the network behaves like an ensemble of shallow networks.
- **Iterative refinement**: Each block computes a small *correction*  $F(\mathbf{h}_\ell)$  to the current representation, rather than building a new one from scratch.
- **Unrolled optimization**: Deep equilibrium models (Bai et al., 2019) show residual blocks can be viewed as steps of an implicit fixed-point iteration:  $\mathbf{h}^* = \mathbf{h}^* + F(\mathbf{h}^*)$ .

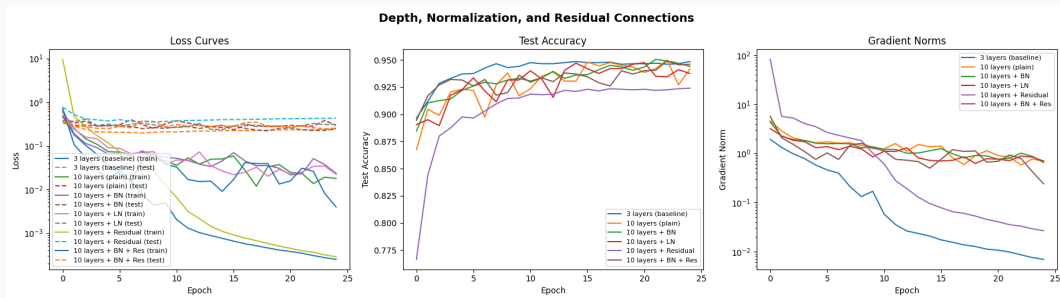
### 3. Residual Connections — Deeper Interpretations

Beyond “gradient highway,” residual connections have richer interpretations:

- **Ensemble of paths** (Veit et al., 2016): A ResNet of  $L$  blocks has  $2^L$  paths from input to output. Empirically, most gradient signal flows through *short* paths — the network behaves like an ensemble of shallow networks.
- **Iterative refinement**: Each block computes a small *correction*  $F(\mathbf{h}_\ell)$  to the current representation, rather than building a new one from scratch.
- **Unrolled optimization**: Deep equilibrium models (Bai et al., 2019) show residual blocks can be viewed as steps of an implicit fixed-point iteration:  $\mathbf{h}^* = \mathbf{h}^* + F(\mathbf{h}^*)$ .

**References:** Veit et al., “Residual Networks Behave Like Ensembles of Relatively Shallow Networks” (NeurIPS 2016); Bai et al., “Deep Equilibrium Models” (NeurIPS 2019).

### 3. Depth, Normalization & Residuals — Lab Results



The plain 10-layer net **degrades** below the shallow 3-layer baseline (the “degradation problem”). Adding **BatchNorm + residuals** recovers and surpasses it — reaching the highest accuracy. Right panel: normalization keeps gradient norms stable across depth.

## 4. Optimizers: Beyond Vanilla SGD

### SGD + Momentum

$$v_t = \beta v_{t-1} + \nabla_{\theta} \mathcal{L}$$

$$\theta_t = \theta_{t-1} - \eta v_t$$

Smooths noisy gradients; accelerates along consistent directions.

### Adam / AdamW

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Per-parameter adaptive LR. Robust default.

## 4. Optimizers: Beyond Vanilla SGD

### SGD + Momentum

$$v_t = \beta v_{t-1} + \nabla_{\theta} \mathcal{L}$$

$$\theta_t = \theta_{t-1} - \eta v_t$$

Smooths noisy gradients; accelerates along consistent directions.

### Adam / AdamW

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Per-parameter adaptive LR. Robust default.

### Rule of thumb

Starting out? AdamW,  $\eta = 10^{-3}$ .

Transformers? AdamW.

Vision SOTA? SGD + momentum can edge ahead with careful tuning.

## 5. Learning Rate: The Most Important Hyperparameter

Too high → divergence

Just right → smooth  
convergence

Too low → painfully slow

**First thing to sweep:**

$$\eta \in \{10^{-4}, 3 \times 10^{-4}, 10^{-3}, 3 \times 10^{-3}\}$$

**Common schedules:**

---

|            |                                   |
|------------|-----------------------------------|
| Step decay | Drop by $\gamma$ every $N$ epochs |
|------------|-----------------------------------|

|        |                                                                                           |
|--------|-------------------------------------------------------------------------------------------|
| Cosine | $\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos \frac{\pi t}{T})$ |
|--------|-------------------------------------------------------------------------------------------|

|                |                                  |
|----------------|----------------------------------|
| Warmup + decay | Ramp up, then cosine/linear down |
|----------------|----------------------------------|

---

**Why warmup?** Early gradients are high-variance (random weights). Low initial LR prevents destructive updates.

## 5. Learning Rate: The Most Important Hyperparameter

Too high → divergence

Just right → smooth  
convergence

Too low → painfully slow

**First thing to sweep:**

$$\eta \in \{10^{-4}, 3 \times 10^{-4}, 10^{-3}, 3 \times 10^{-3}\}$$

**Common schedules:**

---

|            |                                   |
|------------|-----------------------------------|
| Step decay | Drop by $\gamma$ every $N$ epochs |
|------------|-----------------------------------|

|        |                                                                                           |
|--------|-------------------------------------------------------------------------------------------|
| Cosine | $\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos \frac{\pi t}{T})$ |
|--------|-------------------------------------------------------------------------------------------|

|                |                                  |
|----------------|----------------------------------|
| Warmup + decay | Ramp up, then cosine/linear down |
|----------------|----------------------------------|

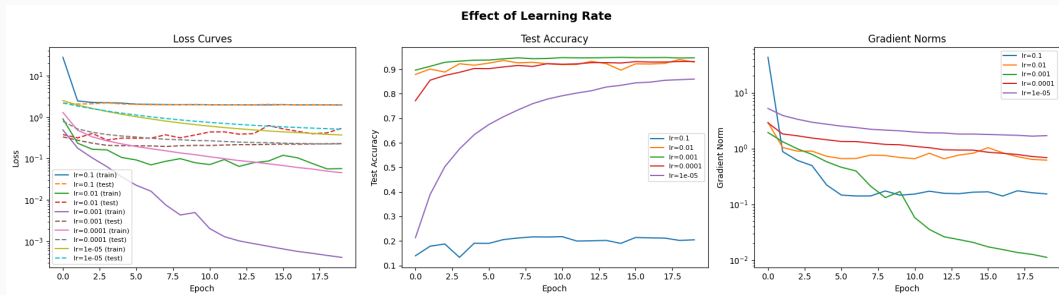
---

**Why warmup?** Early gradients are high-variance (random weights). Low initial LR prevents destructive updates.

**Why decay is necessary**

Convergence theory (Robbins–Monro) requires  $\sum \eta_t = \infty$  (reach the optimum) and  $\sum \eta_t^2 < \infty$  (don't oscillate past it). A constant LR violates the second condition — you *must* decay.

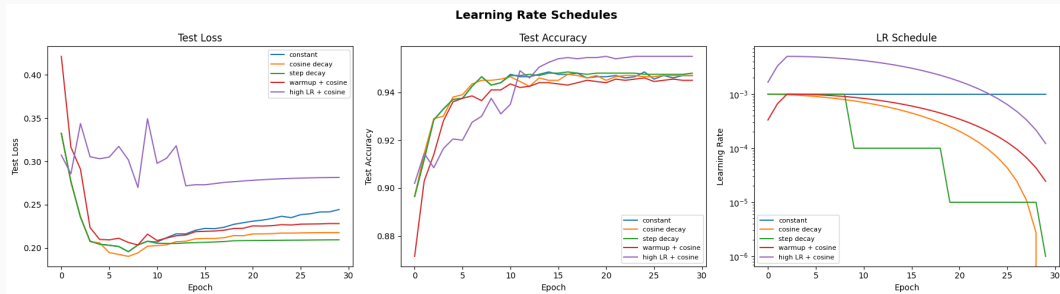
## 5. Learning Rate — Lab Results



$\eta = 10^{-1}$ : **diverges** (loss spikes, accuracy crashes).  $\eta = 10^{-5}$ : **too slow** (barely learns in 20 epochs).  
 $\eta \in \{10^{-2}, 10^{-3}\}$ : **sweet spot** — fast convergence, high accuracy.

❖ **Lab §6** — LR sweep: trains with  $\eta \in \{10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}\}$ .

## 5. Learning Rate Schedules — Lab Results



Cosine and step decay outperform constant LR on test loss. Warmup + cosine is most stable early on. Right panel: the actual LR trajectory for each schedule.

❖ **Lab §7** — Schedule comparison: constant vs. cosine vs. step vs. warmup + cosine.

## 6. The Learning Rate — Batch Size Relationship

### Linear Scaling Rule (Goyal et al., 2017)

When you multiply batch size by  $k$ , multiply learning rate by  $k$ :

$$\eta_{\text{new}} = \eta_{\text{base}} \times \frac{B_{\text{new}}}{B_{\text{base}}}$$

**Intuition:** larger batch  $\rightarrow$  lower-variance gradient  $\rightarrow$  can take bigger steps.

❖ **Lab §8** — Batch size experiment: compares baseline, wrong (same LR), correctly scaled, and scaled down.

## 6. The Learning Rate — Batch Size Relationship

### Linear Scaling Rule (Goyal et al., 2017)

When you multiply batch size by  $k$ , multiply learning rate by  $k$ :

$$\eta_{\text{new}} = \eta_{\text{base}} \times \frac{B_{\text{new}}}{B_{\text{base}}}$$

**Intuition:** larger batch  $\rightarrow$  lower-variance gradient  $\rightarrow$  can take bigger steps.

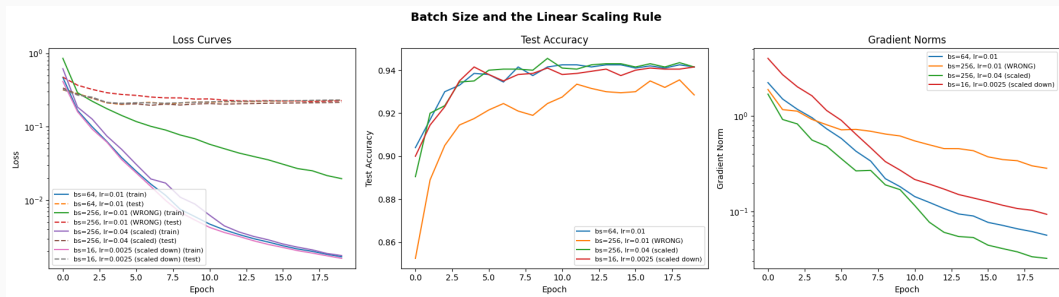
### Practical implication

If you change GPU and therefore batch size, **always** rescale LR.

Gradient accumulation changes effective batch size too.

❖ **Lab §8** — Batch size experiment: compares baseline, wrong (same LR), correctly scaled, and scaled down.

## 6. Batch Size & LR Scaling — Lab Results



**$bs=256, lr=0.001$  (WRONG):** fewer steps per epoch *and* same step size  $\rightarrow$  under-trains.  **$bs=256, lr=0.004$  (scaled):**  $4\times$  larger LR compensates for  $4\times$  fewer steps — matches the baseline. The scaled-down run ( $bs=16$ ) also tracks correctly.

## 7. Gradient Clipping

**Problem:** gradient norms can **spike** → destructive parameter updates.

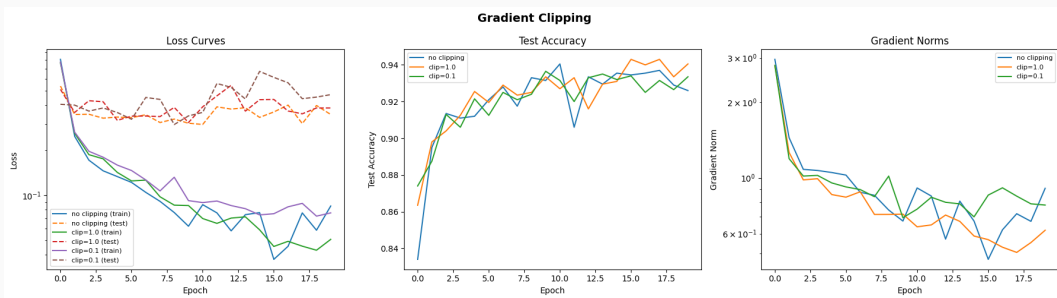
**Fix — norm clipping:**

$$\nabla_{\theta} \mathcal{L} \leftarrow \begin{cases} \nabla_{\theta} \mathcal{L} & \text{if } \|\nabla_{\theta} \mathcal{L}\| \leq c \\ c \cdot \frac{\nabla_{\theta} \mathcal{L}}{\|\nabla_{\theta} \mathcal{L}\|} & \text{otherwise} \end{cases}$$

- Preserves gradient **direction**; bounds magnitude.
- Standard for transformers ( $c = 1.0$ ).
- A **safety net**, not a crutch — shouldn't fire often if other choices are good.

❖ **Lab §9** — Clipping experiment with a deliberately unstable setup (deep, no norm, high LR).

## 7. Gradient Clipping — Lab Results



Without clipping, gradient norms are volatile (right panel) and training is unstable. **clip=1.0** tames the worst spikes while preserving learning speed. **clip=0.1** is too aggressive — gradients are over-dampened and accuracy lags behind.

## 8. Gradient Accumulation

**Problem:** you want effective batch size 256 but your GPU fits 64.

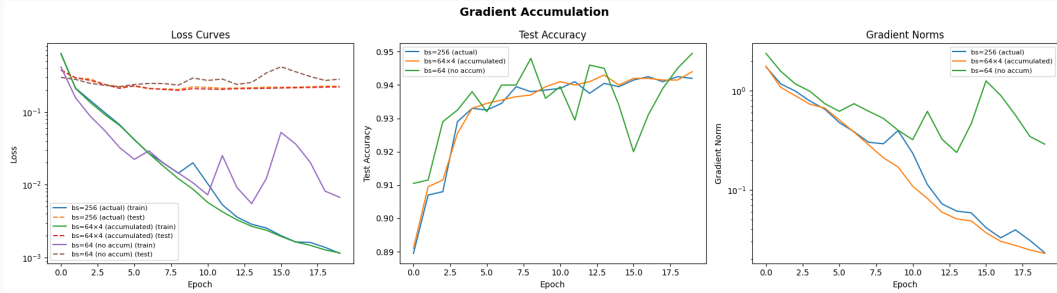
**Solution:** accumulate gradients over  $k$  micro-batches before updating.

```
optimizer.zero_grad()
for i in range(accumulation_steps):           # e.g. 4
    loss = model(micro_batch[i]) / accumulation_steps
    loss.backward()                           # grads accumulate in .grad
optimizer.step()                             # one update
```

$$B_{\text{eff}} = B_{\text{micro}} \times k_{\text{accum}} \times n_{\text{GPUs}}$$

❖ **Lab §10** — Shows  $\text{bs}=256$  (actual)  $\approx \text{bs}=64 \times 4$  accum (simulated)  $\neq \text{bs}=64$  (no accum).

## 8. Gradient Accumulation — Lab Results



The **accumulated** run (orange,  $bs=64 \times 4$ ) closely tracks the **actual  $bs=256$**  run (blue) in both loss and accuracy. Plain  $bs=64$  without accumulation (green) follows a different trajectory — more parameter updates per epoch with noisier gradients.

## 9. Monitoring: You Can't Optimize What You Can't See

### What to track (wandb, TensorBoard)

| Metric          | Signal                       |
|-----------------|------------------------------|
| Train loss      | Is optimization working?     |
| Val loss        | Is the model generalizing?   |
| Train-val gap   | Overfitting?                 |
| Gradient norms  | Vanishing / exploding?       |
| Learning rate   | Schedule behaving correctly? |
| GPU utilization | Pipeline efficient?          |

### Red flags

Loss flat — LR too low? Bug?

Loss NaN — exploding grads

Train ↓, val ↑ — overfit

Grad norm 0 — vanishing

Grad norm  $\rightarrow \infty$  — exploding

❖ **Lab §11** — Toggle `USE_WANDB = True`; logs train/val loss, accuracy, gradient norms, and LR to a live dashboard.

## 10. Regularization (Brief)

Especially important for **small health datasets**:

**Weight decay** L2 penalty on parameters. Built into AdamW.

**Dropout** Randomly zero neurons during training. Forces redundancy.

**Early stopping** Halt when val loss plateaus. Simple and effective.

**Data augmentation** Synthetic variety (rotations, crops, masking). Especially for images.

## 10. Regularization (Brief)

Especially important for **small health datasets**:

**Weight decay** L2 penalty on parameters. Built into AdamW.

**Dropout** Randomly zero neurons during training. Forces redundancy.

**Early stopping** Halt when val loss plateaus. Simple and effective.

**Data augmentation** Synthetic variety (rotations, crops, masking). Especially for images.

### Rule of thumb

Always use weight decay (AdamW default).

Overfitting? Add dropout (0.1–0.5) and/or early stopping.

Images? Data augmentation is almost always worth it.

## Putting It All Together

---

# A Training Recipe

1. Choose architecture (match modality)
2. Initialize properly (Kaiming / Xavier)
3. Add normalization + residual connections
4. Start with **AdamW**,  $\eta = 10^{-3}$
5. LR warmup (5–10% of training)
6. Cosine or linear LR decay
7. Gradient clipping (max norm 1.0)
8. Gradient accumulation if memory-limited
9. **Monitor everything** with wandb
10. Evaluate on held-out data; early stop

# Summary

- Neural networks = **compositions of simple differentiable ops** — the simplest function class satisfying universality + parametric + autodiff.

# Summary

- Neural networks = **compositions of simple differentiable ops** — the simplest function class satisfying universality + parametric + autodiff.
- Core unit:  $\sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$ ; depth  $\times$  width = expressiveness.

# Summary

- Neural networks = **compositions of simple differentiable ops** — the simplest function class satisfying universality + parametric + autodiff.
- Core unit:  $\sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$ ; depth  $\times$  width = expressiveness.
- **Architecture encodes inductive bias**: conv  $\rightarrow$  locality; attention  $\rightarrow$  learned interactions; MLP  $\rightarrow$  no structural prior.

# Summary

- Neural networks = **compositions of simple differentiable ops** — the simplest function class satisfying universality + parametric + autodiff.
- Core unit:  $\sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$ ; depth  $\times$  width = expressiveness.
- **Architecture encodes inductive bias**: conv  $\rightarrow$  locality; attention  $\rightarrow$  learned interactions; MLP  $\rightarrow$  no structural prior.
- Practical optimization requires **init + normalization + residuals + LR schedule + monitoring**.

# Summary

- Neural networks = **compositions of simple differentiable ops** — the simplest function class satisfying universality + parametric + autodiff.
- Core unit:  $\sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$ ; depth  $\times$  width = expressiveness.
- **Architecture encodes inductive bias**: conv  $\rightarrow$  locality; attention  $\rightarrow$  learned interactions; MLP  $\rightarrow$  no structural prior.
- Practical optimization requires **init + normalization + residuals + LR schedule + monitoring**.
- As the lab results show: each technique has a visible, measurable impact on training dynamics.

# Summary

- Neural networks = **compositions of simple differentiable ops** — the simplest function class satisfying universality + parametric + autodiff.
- Core unit:  $\sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$ ; depth  $\times$  width = expressiveness.
- **Architecture encodes inductive bias**: conv  $\rightarrow$  locality; attention  $\rightarrow$  learned interactions; MLP  $\rightarrow$  no structural prior.
- Practical optimization requires **init + normalization + residuals + LR schedule + monitoring**.
- As the lab results show: each technique has a visible, measurable impact on training dynamics.

❖ **Notebook:** `nn_fundamentals_lab.ipynb` — reproduce and extend every experiment



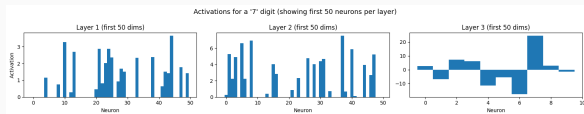
# What the Network Learns: First-Layer Weights



Each  $28 \times 28$  tile is one row of  $W_1$  reshaped to image space — the pattern that neuron responds to. Some show stroke-like or edge features; others are harder to interpret. Unlike CNN filters (local patches), MLP weights span the **entire image** — no locality prior.

# What the Network Learns: Activations & Predictions

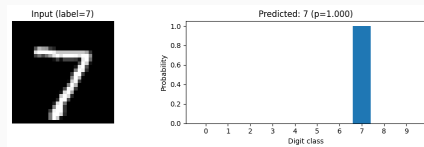
## Per-layer activations for a '7':



Layers 1 and 2 (after ReLU): sparse, non-negative.

Layer 3 (logits): 10 raw values, one per class — both positive and negative.

## Final softmax output:



The network is **confident**:  $p('7') \approx 1.0$ .

This is the full pipeline:

pixels  $\rightarrow$  features  $\rightarrow$  logits  $\rightarrow$  softmax  
 $\rightarrow$  prediction.

# Batch Size & Learning Rate: The Full Picture

The linear scaling rule (slide 6) is a **special case** of a deeper principle.

## The noise scale (Smith & Le, 2017; Smith et al., 2018)

SGD dynamics are governed by a *noise scale*:

$$g = \frac{\varepsilon N}{B} \quad (\text{vanilla SGD}), \quad g = \frac{\varepsilon N}{B(1-m)} \quad (\text{SGD} + \text{momentum})$$

where  $\varepsilon$  is the learning rate,  $N$  the dataset size,  $B$  the batch size, and  $m$  the momentum coefficient.

**Key insight:** Goyal's rule ( $\varepsilon \propto B$ ) simply holds  $g$  constant. But Smith et al. showed that *decaying* the LR is equivalent to *increasing* the batch size — both reduce  $g$ .

## Implication

LR schedules are really **noise annealing** schedules. You can convert any LR decay into an equivalent batch-size increase — same learning curves, fewer parameter updates.

# The Linear Scaling Rule Breaks Down for Adam

The Goyal et al. linear rule ( $\epsilon \propto B$ ) was derived for **SGD**. Modern transformer training uses **AdamW**, where the dynamics are fundamentally different.

| Optimizer          | Scaling rule                              | Source               |
|--------------------|-------------------------------------------|----------------------|
| SGD / SGD+momentum | $\epsilon \propto B$ (linear)             | Goyal et al., 2017   |
| Adam / AdamW       | $\epsilon \propto \sqrt{B}$ (square root) | Malladi et al., 2022 |

**Why?** Adam normalizes updates by the second moment of gradients. Increasing  $B$  reduces gradient variance, but Adam's denominator partially *compensates* for this automatically — so you need a smaller LR adjustment than with SGD.

# The Linear Scaling Rule Breaks Down for Adam

The Goyal et al. linear rule ( $\varepsilon \propto B$ ) was derived for **SGD**. Modern transformer training uses **AdamW**, where the dynamics are fundamentally different.

| Optimizer          | Scaling rule                                 | Source               |
|--------------------|----------------------------------------------|----------------------|
| SGD / SGD+momentum | $\varepsilon \propto B$ (linear)             | Goyal et al., 2017   |
| Adam / AdamW       | $\varepsilon \propto \sqrt{B}$ (square root) | Malladi et al., 2022 |

**Why?** Adam normalizes updates by the second moment of gradients. Increasing  $B$  reduces gradient variance, but Adam's denominator partially *compensates* for this automatically — so you need a smaller LR adjustment than with SGD.

## The “surge” phenomenon (Li et al., NeurIPS 2024)

For Adam, the optimal LR is **non-monotonic** in  $B$ : it first rises, then *falls*, then converges to a constant. At very large  $B$ , Adam's update  $\approx \text{sign}(\nabla \mathcal{L})$ , which **saturates** — adding more samples per step doesn't help.

# Critical Batch Size: When Bigger Stops Helping

## Critical Batch Size (McCandlish et al., 2018)

There exists a **critical batch size**  $B^*$  such that:

- $B < B^*$ : doubling  $B$  roughly halves training steps (linear speedup).
- $B > B^*$ : diminishing returns — you pay more compute for marginal gains.

## What determines $B^*$ ?

- $B^*$  **grows during training** — near zero at init, increases rapidly, then plateaus (Merrill et al., 2025). This naturally motivates **batch size warmup**.
- $B^*$  scales with **dataset size** more than model size (Zhang et al., 2024).
- For LLMs,  $B^*$  can be very large ( $10^4$ – $10^6$  tokens), but McCandlish's gradient noise scale proxy is unreliable under Adam (Merrill et al., 2025).

# Critical Batch Size: When Bigger Stops Helping

## Critical Batch Size (McCandlish et al., 2018)

There exists a **critical batch size**  $B^*$  such that:

- $B < B^*$ : doubling  $B$  roughly halves training steps (linear speedup).
- $B > B^*$ : diminishing returns — you pay more compute for marginal gains.

## What determines $B^*$ ?

- $B^*$  **grows during training** — near zero at init, increases rapidly, then plateaus (Merrill et al., 2025). This naturally motivates **batch size warmup**.
- $B^*$  scales with **dataset size** more than model size (Zhang et al., 2024).
- For LLMs,  $B^*$  can be very large ( $10^4$ – $10^6$  tokens), but McCandlish's gradient noise scale proxy is unreliable under Adam (Merrill et al., 2025).

## Practical upshot

Don't blindly maximize  $B$ . Use the largest batch size **below**  $B^*$  for your problem, scale LR accordingly, and consider ramping  $B$  up during training.

# Putting It Together: Batch Size Strategy for Transformers

## What the main slides teach (correct!):

- If you change  $B$ , rescale  $\varepsilon$
- Grad accumulation = larger effective  $B$
- Warmup + cosine is a good default

## The fuller picture:

- LR decay  $\equiv$  noise annealing  $\equiv$  batch size increase (Smith et al., 2018)
- Use  $\varepsilon \propto \sqrt{B}$  for AdamW, not linear
- There is a critical  $B^*$  — past it, you waste compute
- $B^*$  grows during training  $\Rightarrow$  batch size warmup

## Modern LLM recipe:

1. Start with small  $B$  (high noise = exploration)
2. Ramp  $B$  up over early training
3. Cap at  $B^*$  or hardware limit
4. Use warmup + cosine LR decay on top
5. This is *two* annealing knobs, not one

## Health AI context

For small clinical datasets,  $B^*$  is often small. **Linear scaling is fine as a starting heuristic.** For foundation model pre-training on EHR, the transformer-era nuances matter more.

## Further Reading: Batch Size & LR Scaling

**Noise scale** Smith & Le, 2017 — “A Bayesian Perspective on Generalization and SGD”

**Don't decay LR** Smith et al., 2018 — “Don't Decay the Learning Rate, Increase the Batch Size” (ICLR 2018)

**Critical BS** McCandlish et al., 2018 — “An Empirical Model of Large-Batch Training”

**Adam scaling** Malladi et al., 2022 — “On the SDEs and Scaling Rules for Adaptive Gradient Algorithms” (NeurIPS 2022)

**Surge phenom.** Li et al., 2024 — “Surge Phenomenon in Optimal Learning Rate and Batch Size Scaling” (NeurIPS 2024)

**CBS revisited** Merrill et al., 2025 — “Critical Batch Size Revisited” (NeurIPS 2025)

**CBS & data size** Zhang et al., 2024 — “How Does Critical Batch Size Scale in Pre-training?”

# Batch Size & Learning Rate: The Full Picture

The linear scaling rule (slide 6) is a **special case** of a deeper principle.

## The noise scale (Smith & Le, 2017; Smith et al., 2018)

SGD dynamics are governed by a *noise scale*:

$$g = \frac{\varepsilon N}{B} \quad (\text{vanilla SGD}), \quad g = \frac{\varepsilon N}{B(1-m)} \quad (\text{SGD} + \text{momentum})$$

where  $\varepsilon$  is the learning rate,  $N$  the dataset size,  $B$  the batch size, and  $m$  the momentum coefficient.

**Key insight:** Goyal's rule ( $\varepsilon \propto B$ ) simply holds  $g$  constant. But Smith et al. showed that *decaying* the LR is equivalent to *increasing* the batch size — both reduce  $g$ .

## Implication

LR schedules are really **noise annealing** schedules. You can convert any LR decay into an equivalent batch-size increase — same learning curves, fewer parameter updates.

# The Linear Scaling Rule Breaks Down for Adam

The Goyal et al. linear rule ( $\epsilon \propto B$ ) was derived for **SGD**. Modern transformer training uses **AdamW**, where the dynamics are fundamentally different.

| Optimizer          | Scaling rule                              | Source               |
|--------------------|-------------------------------------------|----------------------|
| SGD / SGD+momentum | $\epsilon \propto B$ (linear)             | Goyal et al., 2017   |
| Adam / AdamW       | $\epsilon \propto \sqrt{B}$ (square root) | Malladi et al., 2022 |

**Why?** Adam normalizes updates by the second moment of gradients. Increasing  $B$  reduces gradient variance, but Adam's denominator partially *compensates* for this automatically — so you need a smaller LR adjustment than with SGD.

# The Linear Scaling Rule Breaks Down for Adam

The Goyal et al. linear rule ( $\varepsilon \propto B$ ) was derived for **SGD**. Modern transformer training uses **AdamW**, where the dynamics are fundamentally different.

| Optimizer          | Scaling rule                                 | Source               |
|--------------------|----------------------------------------------|----------------------|
| SGD / SGD+momentum | $\varepsilon \propto B$ (linear)             | Goyal et al., 2017   |
| Adam / AdamW       | $\varepsilon \propto \sqrt{B}$ (square root) | Malladi et al., 2022 |

**Why?** Adam normalizes updates by the second moment of gradients. Increasing  $B$  reduces gradient variance, but Adam's denominator partially *compensates* for this automatically — so you need a smaller LR adjustment than with SGD.

## The “surge” phenomenon (Li et al., NeurIPS 2024)

For Adam, the optimal LR is **non-monotonic** in  $B$ : it first rises, then *falls*, then converges to a constant. At very large  $B$ , Adam's update  $\approx \text{sign}(\nabla \mathcal{L})$ , which **saturates** — adding more samples per step doesn't help.

# Critical Batch Size: When Bigger Stops Helping

## Critical Batch Size (McCandlish et al., 2018)

There exists a **critical batch size**  $B^*$  such that:

- $B < B^*$ : doubling  $B$  roughly halves training steps (linear speedup).
- $B > B^*$ : diminishing returns — you pay more compute for marginal gains.

## What determines $B^*$ ?

- $B^*$  **grows during training** — near zero at init, increases rapidly, then plateaus (Merrill et al., 2025). This naturally motivates **batch size warmup**.
- $B^*$  scales with **dataset size** more than model size (Zhang et al., 2024).
- For LLMs,  $B^*$  can be very large ( $10^4$ – $10^6$  tokens), but McCandlish's gradient noise scale proxy is unreliable under Adam (Merrill et al., 2025).

# Critical Batch Size: When Bigger Stops Helping

## Critical Batch Size (McCandlish et al., 2018)

There exists a **critical batch size**  $B^*$  such that:

- $B < B^*$ : doubling  $B$  roughly halves training steps (linear speedup).
- $B > B^*$ : diminishing returns — you pay more compute for marginal gains.

## What determines $B^*$ ?

- $B^*$  **grows during training** — near zero at init, increases rapidly, then plateaus (Merrill et al., 2025). This naturally motivates **batch size warmup**.
- $B^*$  scales with **dataset size** more than model size (Zhang et al., 2024).
- For LLMs,  $B^*$  can be very large ( $10^4$ – $10^6$  tokens), but McCandlish's gradient noise scale proxy is unreliable under Adam (Merrill et al., 2025).

## Practical upshot

Don't blindly maximize  $B$ . Use the largest batch size **below**  $B^*$  for your problem, scale LR accordingly, and consider ramping  $B$  up during training.

# Putting It Together: Batch Size Strategy for Transformers

## What the main slides teach (correct!):

- If you change  $B$ , rescale  $\varepsilon$
- Grad accumulation = larger effective  $B$
- Warmup + cosine is a good default

## The fuller picture:

- LR decay  $\equiv$  noise annealing  $\equiv$  batch size increase (Smith et al., 2018)
- Use  $\varepsilon \propto \sqrt{B}$  for AdamW, not linear
- There is a critical  $B^*$  — past it, you waste compute
- $B^*$  grows during training  $\Rightarrow$  batch size warmup

## Modern LLM recipe:

1. Start with small  $B$  (high noise = exploration)
2. Ramp  $B$  up over early training
3. Cap at  $B^*$  or hardware limit
4. Use warmup + cosine LR decay on top
5. This is *two* annealing knobs, not one

## Health AI context

For small clinical datasets,  $B^*$  is often small. **Linear scaling is fine as a starting heuristic.** For foundation model pre-training on EHR, the transformer-era nuances matter more.

## Further Reading: Batch Size & LR Scaling

**Noise scale** Smith & Le, 2017 — “A Bayesian Perspective on Generalization and SGD”

**Don't decay LR** Smith et al., 2018 — “Don't Decay the Learning Rate, Increase the Batch Size” (ICLR 2018)

**Critical BS** McCandlish et al., 2018 — “An Empirical Model of Large-Batch Training”

**Adam scaling** Malladi et al., 2022 — “On the SDEs and Scaling Rules for Adaptive Gradient Algorithms” (NeurIPS 2022)

**Surge phenom.** Li et al., 2024 — “Surge Phenomenon in Optimal Learning Rate and Batch Size Scaling” (NeurIPS 2024)

**CBS revisited** Merrill et al., 2025 — “Critical Batch Size Revisited” (NeurIPS 2025)

**CBS & data size** Zhang et al., 2024 — “How Does Critical Batch Size Scale in Pre-training?”